

The vegas routine performs $m = \text{itm}$ x statistically independent evaluations of the desired integral, each with $N = \text{nca}$ ll function evaluations. While statistically independent, these iterations do assist each other, since each one is used to refine the sampling grid for the next one. The results of all iterations are combined into a single best answer, and its estimated error, by the relations

$$I_{\text{best}} = \sum_{i=0}^{m-1} \frac{I_i}{\sigma_i^2} / \sum_{i=0}^{m-1} \frac{1}{\sigma_i^2} \quad \sigma_{\text{best}} = \left(\sum_{i=0}^{m-1} \frac{1}{\sigma_i^2} \right)^{-1/2} \quad (7.9.18)$$

Also returned is the quantity

$$\chi^2/m \equiv \frac{1}{m-1} \sum_{i=0}^{m-1} \frac{(I_i - I_{\text{best}})^2}{\sigma_i^2} \quad (7.9.19)$$

If this is significantly larger than 1, then the results of the iterations are statistically inconsistent, and the answers are suspect.

Here is the interface to vegas. (The full code is given in [3].)

```
void vegas(VecDoub_I &regn, Doub fxn(VecDoub_I &, const Doub), const Int init,
          const Int ncall, const Int itm, const Int nprn, Doub &tgral, Doub &sd,
          Doub &chi2a) {
```

Performs Monte Carlo integration of a user-supplied ndim -dimensional function fxn over a rectangular volume specified by $\text{regn}[0..2*\text{ndim}-1]$, a vector consisting of ndim “lower left” coordinates of the region followed by ndim “upper right” coordinates. The integration consists of itm x iterations, each with approximately nca ll calls to the function. After each iteration the grid is refined; more than 5 or 10 iterations are rarely useful. The input flag init signals whether this call is a new start or a subsequent call for additional iterations (see comments in the code). The input flag nprn (normally 0) controls the amount of diagnostic output. Returned answers are tgral (the best estimate of the integral), sd (its standard deviation), and chi2a (χ^2 per degree of freedom, an indicator of whether consistent results are being obtained). See text for further details.

The input flag init can be used to advantage. One might have a call with $\text{init}=0$, nca ll=1000, itm x=5 immediately followed by a call with $\text{init}=1$, nca ll=100000, itm x=1. The effect would be to develop a sampling grid over five iterations of a small number of samples, then to do a single high accuracy integration on the optimized grid.

To use vegas for the torus example discussed in §7.7 (the density integrand only, say), the function fxn would be

```
Doub torusfunc(const VecDoub &x, const Doub wgt) {
    Doub den = exp(5.*x[2]);
    if (SQR(x[2])+SQR(sqrt(SQR(x[0])+SQR(x[1]))-3.) <= 1.) return den;
    else return 0.;
}
```

and the main code would be

```
Doub tgral, sd, chi2a;
VecDoub regn(6);
regn[0] = 1.; regn[3] = 4.;
regn[1] = -3.; regn[4] = 4.;
regn[2] = -1.; regn[5] = 1.;
vegas(regn,torusfunc,0,10000,10,0,tgral,sd,chi2a);
vegas(regn,torusfunc,1,900000,1,0,tgral,sd,chi2a);
```

Note that the user-supplied integrand function, fxn , has an argument wgt in addition to the expected evaluation point $\mathbf{x}$. In most applications you ignore wgt inside the function. Occasionally, however, you may want to integrate some additional function or functions along with the principal function f . The integral of any such function g can be estimated by

$$I_g = \sum_i w_i g(\mathbf{x}) \quad (7.9.20)$$

where the w_i 's and $\mathbf{x}$'s are the arguments `wgt` and `x`, respectively. It is straightforward to accumulate this sum inside your function `fxn` and to pass the answer back to your main program via global variables. Of course, $g(\mathbf{x})$ had better resemble the principal function f to some degree, since the sampling will be optimized for f .

The full listing of `vegas` is given in a Webnote [3].

7.9.5 Recursive Stratified Sampling

The problem with stratified sampling, we have seen, is that it may not avoid the K^d explosion inherent in the obvious, Cartesian, tessellation of a d -dimensional volume. A technique called *recursive stratified sampling* [4] attempts to do this by successive bisections of a volume, not along all d dimensions, but rather along only one dimension at a time. The starting points are equations (7.9.10) and (7.9.13), applied to bisections of successively smaller subregions.

Suppose that we have a quota of N evaluations of the function f and want to evaluate $\langle f \rangle'$ in the rectangular parallelepiped region $R = (\mathbf{x}_a, \mathbf{x}_b)$. (We denote such a region by the two coordinate vectors of its diagonally opposite corners.) First, we allocate a fraction p of N toward exploring the variance of f in R : We sample pN function values uniformly in R and accumulate the sums that will give the d different pairs of variances corresponding to the d different coordinate directions along which R can be bisected. In other words, in pN samples, we estimate $\text{Var}(f)$ in each of the regions resulting from a possible bisection of R ,

$$\begin{aligned} R_{ai} &\equiv (\mathbf{x}_a, \mathbf{x}_b - \tfrac{1}{2}\mathbf{e}_i \cdot (\mathbf{x}_b - \mathbf{x}_a)\mathbf{e}_i) \\ R_{bi} &\equiv (\mathbf{x}_a + \tfrac{1}{2}\mathbf{e}_i \cdot (\mathbf{x}_b - \mathbf{x}_a)\mathbf{e}_i, \mathbf{x}_b) \end{aligned} \quad (7.9.21)$$

Here $\mathbf{e}_i$ is the unit vector in the i th coordinate direction, $i = 1, 2, \dots, d$.

Second, we inspect the variances to find the most favorable dimension i to bisect. By equation (7.9.15), we could, for example, choose that i for which the sum of the square roots of the variance estimators in regions R_{ai} and R_{bi} is minimized. (Actually, as we will explain, we do something slightly different.)

Third, we allocate the remaining $(1-p)N$ function evaluations between the regions R_{ai} and R_{bi} . If we used equation (7.9.15) to choose i , we should do this allocation according to equation (7.9.14).

We now have two parallelepipeds, each with its own allocation of function evaluations for estimating the mean of f . Our “RSS” algorithm now shows itself to be *recursive*: To evaluate the mean in each region, we go back to the sentence beginning “First,...” in the paragraph above equation (7.9.21). (Of course, when the allocation of points to a region falls below some number, we resort to simple Monte Carlo rather than continue with the recursion.)

Finally, we combine the means and also estimated variances of the two subvolumes using equation (7.9.10) and the first line of equation (7.9.11).

This completes the RSS algorithm in its simplest form. Before we describe some additional tricks under the general rubric of “implementation details,” we need to return briefly to equations (7.9.13) – (7.9.15) and derive the equations that we actually use instead of these. The right-hand side of equation (7.9.13) applies the familiar scaling law of equation (7.9.9) twice, once to a and again to b . This would be correct if the estimates $\langle f \rangle_a$ and $\langle f \rangle_b$ were each made by simple Monte Carlo, with uniformly random sample points. However, the two estimates of the mean are in fact made recursively. Thus, there is no reason to expect equation (7.9.9) to hold. Rather, we might substitute for equation (7.9.13) the relation,

$$\text{Var}(\langle f \rangle') = \frac{1}{4} \left[\frac{\text{Var}_a(f)}{N_a^\alpha} + \frac{\text{Var}_b(f)}{(N - N_a)^\alpha} \right] \quad (7.9.22)$$

where α is an unknown constant ≥ 1 (the case of equality corresponding to simple Monte Carlo). In that case, a short calculation shows that $\text{Var}(\langle f \rangle')$ is minimized when

$$\frac{N_a}{N} = \frac{\text{Var}_a(f)^{1/(1+\alpha)}}{\text{Var}_a(f)^{1/(1+\alpha)} + \text{Var}_b(f)^{1/(1+\alpha)}} \quad (7.9.23)$$

and that its minimum value is

$$\text{Var}(\langle f \rangle') \propto \left[\text{Var}_a(f)^{1/(1+\alpha)} + \text{Var}_b(f)^{1/(1+\alpha)} \right]^{1+\alpha} \quad (7.9.24)$$

Equations (7.9.22) – (7.9.24) reduce to equations (7.9.13) – (7.9.15) when $\alpha = 1$. Numerical experiments to find a self-consistent value for α find that $\alpha \approx 2$. That is, when equation (7.9.23) with $\alpha = 2$ is used recursively to allocate sample opportunities, the observed variance of the RSS algorithm goes approximately as N^{-2} , while any other value of α in equation (7.9.23) gives a poorer fall-off. (The sensitivity to α is, however, not very great; it is not known whether $\alpha = 2$ is an analytically justifiable result or only a useful heuristic.)

The principal difference between `miser`'s implementation and the algorithm as described thus far lies in how the variances on the right-hand side of equation (7.9.23) are estimated. We find empirically that it is somewhat more robust to use the square of the difference of maximum and minimum sampled function values, instead of the genuine second moment of the samples. This estimator is of course increasingly biased with increasing sample size; however, equation (7.9.23) uses it only to compare two subvolumes (a and b) having approximately equal numbers of samples. The “max minus min” estimator proves its worth when the preliminary sampling yields only a single point, or a small number of points, in active regions of the integrand. In many realistic cases, these are indicators of nearby regions of even greater importance, and it is useful to let them attract the greater sampling weight that “max minus min” provides.

A second modification embodied in the code is the introduction of a “dithering parameter,” `dith`, whose nonzero value causes subvolumes to be divided not exactly down the middle, but rather into fractions $0.5 \pm \text{dith}$, with the sign of the $\pm$ randomly chosen by a built-in random number routine. Normally `dith` can be set to zero. However, there is a large advantage in taking `dith` to be nonzero if some special symmetry of the integrand puts the active region exactly at the midpoint of the region, or at the center of some power-of-two submultiple of the region. One wants to avoid the extreme case of the active region being evenly divided into 2^d abutting corners of a d -dimensional space. A typical nonzero value of `dith`, on those occasions when it is useful, might be 0.1. Of course, when the dithering parameter is nonzero, we must take the differing sizes of the subvolumes into account; the code does this through the variable `frac1`.

One final feature in the code deserves mention. The RSS algorithm uses a single set of sample points to evaluate equation (7.9.23) in all d directions. At the bottom levels of the recursion, the number of sample points can be quite small. Although rare, it can happen that in one direction all the samples are in one half of the volume; in that case, that direction is ignored as a candidate for bifurcation. Even more rare is the possibility that all of the samples are in one half of the volume in *all* directions. In this case, a random direction is chosen. If this happens too often in your application, then you should increase MNPT (see line `if (jb == -1)...` in the code).

Note that `miser`, as given, returns as `ave` an estimate of the average function value $\langle f \rangle$, not the integral of f over the region. The routine `vegas`, adopting the other convention, returns as `tgral` the integral. The two conventions are of course trivially related, by equation (7.9.8), since the volume V of the rectangular region is known.

The interface to the `miser` routine is this:

```
void miser(Doub func(VecDoub_I &), VecDoub_I &regn, const Int npts,
           const Doub dith, Doub &ave, Doub &var) {
```

Monte Carlo samples a user-supplied `ndim`-dimensional function `func` in a rectangular volume specified by `regn[0..2*ndim-1]`, a vector consisting of `ndim` “lower-left” coordinates of the region followed by `ndim` “upper-right” coordinates. The function is sampled a total of `npts` times, at locations determined by the method of recursive stratified sampling. The mean value of the function in the region is returned as `ave`; an estimate of the statistical uncertainty of `ave` (square of standard deviation) is returned as `var`. The input parameter `dith` should normally be set to zero, but can be set to (e.g.) 0.1 if `func`'s active region falls on the boundary of a power-of-two subdivision of region.

Implementing code for the torus problem in §7.7 is